

Министерство науки и высшего образования Российской Федерации
федеральное государственное автономное образовательное учреждение высшего
образования
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»

Отчет

по Лабораторной работе №6

по дисциплине «Практика и Реализация Баз Данных»

Автор: Конанчук Иван Алексеевич

Факультет: ФПИИ

Группа: К3240



Перед выполнением были установлены и протестированны mongodb community edition и mongosh.

1. Создание коллекции

Выбираем базу данны learn, вставляем всех единорогов кроме Dunx первым способом:

```
test> use learn
switched to db learn
learn> db.unicorns.insertMany([
  {name: "Horny", loves: ["carrot", "papaya"], weight: 600, gender: "m", vampires: 63},
  {name: "Aurora", loves: ["carrot", "grape"], weight: 450, gender: "f", vampires: 43},
  {name: "Unicrom", loves: ["energion", "redbull"], weight: 984, gender: "m", vampires: 182},
  {name: "Rooodoodles", loves: ["apple"], weight: 575, gender: "m", vampires: 99},
  {name: "Solnara", loves: ["apple", "carrot", "chocolate"], weight: 550, gender: "f", vampires: 80},
  {name: "Ayna", loves: ["strawberry", "lemon"], weight: 733, gender: "f", vampires: 40},
  {name: "Kenny", loves: ["grape", "lemon"], weight: 690, gender: "m", vampires: 39},
  {name: "Raleigh", loves: ["apple", "sugar"], weight: 421, gender: "m", vampires: 2},
  {name: "Leia", loves: ["apple", "watermelon"], weight: 601, gender: "f", vampires: 33},
  {name: "Pilot", loves: ["apple", "watermelon"], weight: 650, gender: "m", vampires: 54},
  {name: "Nimue", loves: ["grape", "carrot"], weight: 540, gender: "f"}
])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6a184bf123a5faabc9abc115'),
    '1': ObjectId('6a184bf123a5faabc9abc116'),
    '2': ObjectId('6a184bf123a5faabc9abc117'),
    '3': ObjectId('6a184bf123a5faabc9abc118'),
    '4': ObjectId('6a184bf123a5faabc9abc119'),
    '5': ObjectId('6a184bf123a5faabc9abc11a'),
    '6': ObjectId('6a184bf123a5faabc9abc11b'),
    '7': ObjectId('6a184bf123a5faabc9abc11c'),
    '8': ObjectId('6a184bf123a5faabc9abc11d'),
    '9': ObjectId('6a184bf123a5faabc9abc11e'),
    '10': ObjectId('6a184bf123a5faabc9abc11f')
  }
}
```

A Dunx отдельно вторым способом:

```
learn> db.unicorns.insertOne({
  name: "Dunx",
  loves: ["grape", "watermelon"],
  weight: 704,
  gender: "m",
  vampires: 165
})
{
  acknowledged: true,
  insertedId: ObjectId('6a184c2023a5faabc9abc120')
}
```

```
learn> db.unicorns.countDocuments()
12
```

2. Выборка из данных

Сортировка самцов по имени:

```
learn> db.unicorns.find({gender: "m"}).sort({name: 1})
[
  {
    _id: ObjectId('6a184c2023a5faabc9abc120'),
    name: 'Dunx',
    loves: [ 'grape', 'watermelon' ],
    weight: 704,
    gender: 'm',
    vampires: 165
  },
  {
    _id: ObjectId('6a184bf123a5faabc9abc115'),
    name: 'Horny',
    loves: [ 'carrot', 'papaya' ],
    weight: 600,
    gender: 'm',
    vampires: 63
  },
  {
    _id: ObjectId('6a184bf123a5faabc9abc11b'),
    name: 'Kenny',
    loves: [ 'grape', 'lemon' ],
    weight: 690,
    gender: 'm',
    vampires: 39
  }
]
```

Сортировка самок единорогов:

```
learn> db.unicorns.find({gender: "f"})
[
  {
    _id: ObjectId('6a184bf123a5faabc9abc116'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  },
  {
    _id: ObjectId('6a184bf123a5faabc9abc119'),
    name: 'Solnara',
    loves: [ 'apple', 'carrot', 'chocolate' ],
    weight: 550,
    gender: 'f'
  }
]
```

Первые три самки:

```
learn> db.unicorns.find({gender: "f"}).limit(3)
[
  {
    _id: ObjectId('6a184bf123a5faabc9abc116'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  },
  {
    _id: ObjectId('6a184bf123a5faabc9abc119'),
    name: 'Solnara',
    loves: [ 'apple', 'carrot', 'chocolate' ],
    weight: 550,
    gender: 'f',
    vampires: 80
  },
  {
    _id: ObjectId('6a184bf123a5faabc9abc11a'),
    name: 'Ayna',
    loves: [ 'strawberry', 'lemon' ],
    weight: 733,
    gender: 'f',
    vampires: 40
  }
]
```

Первая самка, любящая carrot:

```
learn> db.unicorns.findOne({gender: "f", loves: "carrot"})
{
  _id: ObjectId('6a184bf123a5faabc9abc116'),
  name: 'Aurora',
  loves: [ 'carrot', 'grape' ],
  weight: 450,
  gender: 'f',
  vampires: 43
}
```

Самцы со скрытыми предпочтениями и полами:

```
learn> db.unicorns.find({gender: "m"}, {likes: 0, gender: 0})
[
  {
    _id: ObjectId('6a184bf123a5faabc9abc115'),
    name: 'Horny',
    weight: 600,
    vampires: 63
  },
  {
    _id: ObjectId('6a184bf123a5faabc9abc117'),
    name: 'Unicrom',
    weight: 984,
    vampires: 182
  },
  {
    _id: ObjectId('6a184bf123a5faabc9abc118')
  }
]
```

Обратный порядок:

```
learn> db.unicorns.find().sort({$natural: -1})
[
  {
    _id: ObjectId('6a184c2023a5faabc9abc120'),
    name: 'Dunx',
    likes: [ 'grape', 'watermelon' ],
    weight: 704,
    gender: 'm',
    vampires: 165
  },
  {
    _id: ObjectId('6a184bf123a5faabc9abc11f')
  }
]
```

Первое предпочтение каждого без _id:

```
learn> db.unicorns.find(
|   {},
|   { _id: 0, name: 1, loves: { $slice: 1 } }
| )
[
  { name: 'Horny', loves: [ 'carrot' ] },
  { name: 'Aurora', loves: [ 'carrot' ] },
  { name: 'Unicrom', loves: [ 'energon' ] },
  { name: 'Rooooooodles', loves: [ 'apple' ] },
  { name: 'Solnara', loves: [ 'apple' ] },
  { name: 'Ayna', loves: [ 'strawberry' ] },
  { name: 'Kenny', loves: [ 'grape' ] },
  { name: 'Raleigh', loves: [ 'apple' ] },
  { name: 'Leia', loves: [ 'apple' ] },
  { name: 'Pilot', loves: [ 'apple' ] },
  { name: 'Nimue', loves: [ 'grape' ] },
  { name: 'Dunx', loves: [ 'grape' ] }
]
```

3. Логические операторы

Самки весом от 500 до 700 кг без _id:

```
learn> db.unicorns.find(
|   {
|     gender: "f",
|     weight: { $gte: 500, $lte: 700 }
|   },
|   { _id: 0 }
| )
[
  {
    name: 'Solnara',
    loves: [ 'apple', 'carrot', 'chocolate' ],
    weight: 550,
    gender: 'f',
    vampires: 80
  },
  {
    name: 'Leia',
    loves: [ 'apple', 'watermelon' ],
    weight: 601,
    gender: 'f',
    vampires: 33
  },
  {
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  }
]
```

Самцы от 500 кг, любящие grape, lemon:

```
learn> db.unicorns.find(
|   {
|     gender: "m",
|     weight: {$gte: 500},
|     loves: {$all: ["grape", "lemon"]}
|   },
|   {_id: 0}
| )
[
|   {
|     name: 'Kenny',
|     loves: [ 'grape', 'lemon' ],
|     weight: 690,
|     gender: 'm',
|     vampires: 39
|   }
| ]
```

Единороги без поля vampires:

```
learn> db.unicorns.find(
|   {vampires: {$exists: false}},
|   {_id: 0}
| )
[
|   {
|     name: 'Nimue',
|     loves: [ 'grape', 'carrot' ],
|     weight: 540,
|     gender: 'f'
|   }
| ]
```

Упорядоченный список имён самцов и первое предпочтение:

```
learn> db.unicorns.find(
|   {gender: "m"},
|   {_id: 0, name: 1, loves: {$slice: 1}}
| ).sort({name: 1})
[
|   { name: 'Dunx', loves: [ 'grape' ] },
|   { name: 'Horny', loves: [ 'carrot' ] },
|   { name: 'Kenny', loves: [ 'grape' ] },
|   { name: 'Pilot', loves: [ 'apple' ] },
|   { name: 'Raleigh', loves: [ 'apple' ] },
|   { name: 'Rooooooodles', loves: [ 'apple' ] },
|   { name: 'Unicrom', loves: [ 'energon' ] }
| ]
```

4. Вложенные объекты: towns

Создаем коллекцию towns:

```
learn> db.towns.insertMany([
  {
    name: "Punxsutawney",
    population: 6200,
    last_sensus: ISODate("2008-01-31"),
    famous_for: ["phil the groundhog"],
    mayor: {
      name: "Jim Wehrle"
    }
  },
  {
    name: "New York",
    population: 22200000,
    last_sensus: ISODate("2009-07-31"),
    famous_for: ["statue of liberty", "food"],
    mayor: {
      name: "Michael Bloomberg",
      party: "I"
    }
  },
  {
    name: "Portland",
    population: 528000,
    last_sensus: ISODate("2009-07-20"),
    famous_for: ["beer", "food"],
    mayor: {
      name: "Sam Adams",
      party: "D"
    }
  }
])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6a18523e23a5faabc9abc121'),
    '1': ObjectId('6a18523e23a5faabc9abc122'),
    '2': ObjectId('6a18523e23a5faabc9abc123')
  }
}
```

(Исправил опечатку population)

Выбираем города с независимыми мэрами:


```
learn> db.towns.find(
|   {"mayor.party": "I"},
|   {_id: 0, name: 1, mayor: 1}
| )
[
|   { name: 'New York', mayor: { name: 'Michael Bloomberg', party: 'I' } }
| ]
```

Города с беспартийными мэрами:

```
learn> db.towns.find(
|   {"mayor.party": {$exists: false}},
|   {_id: 0, name: 1, mayor: 1}
| )
[ { name: 'Punxsutawney', mayor: { name: 'Jim Wehrle' } } ]
```

5. JS-функция и курсор

Создаем фильтр с JS-условием, курсор, и выводим:

```
learn> var maleFilter = {
|   $where: function() {
|     return this.gender === "m";
|   }
| }

learn> var cursor = db.unicorns.find(
|   maleFilter,
|   {_id: 0, name: 1, gender: 1}
| ).sort({name: 1}).limit(2)

learn> cursor.forEach(function(obj) {
|   printjson(obj)
| })
{
|   name: 'Dunx',
|   gender: 'm'
| }
{
|   name: 'Horny',
|   gender: 'm'
| }
```

6. Агрегированные запросы

Количество самок весом от 500 до 600 кг:

```
learn> db.unicorns.countDocuments({
|   gender: "f",
|   weight: {$gte: 500, $lte: 600}
| })
2
```

Список УНИКАЛЬНЫХ предпочтений:

```
learn> db.unicorns.distinct("loves")
[
  'apple',      'carrot',
  'chocolate', 'energon',
  'grape',      'lemon',
  'papaya',     'redbull',
  'strawberry', 'sugar',
  'watermelon'
]
```

Количество особей обоих полов:

```
learn> db.unicorns.aggregate([
|   {
|     $group: {
|       _id: "$gender",
|       count: {$sum: 1}
|     }
|   }
| ])
[ { _id: 'm', count: 7 }, { _id: 'f', count: 5 } ]
```

7. Изменение данных

Добавляем Barny:

```
learn> db.unicorns.insertOne({
|   name: "Barney",
|   loves: ["grape"],
|   weight: 340,
|   gender: "m"
| })
|
| db.unicorns.find({name: "Barney"})
|
| [
|   {
|     _id: ObjectId('6a18540923a5faabc9abc124'),
|     name: 'Barney',
|     loves: [ 'grape' ],
|     weight: 340,
|     gender: 'm'
|   }
| ]
```

Изменяем Айна:

```
learn> db.unicorns.updateOne(
|   {name: "Ayna"},
|   {$set: {weight: 800, vampires: 51}}
| )
|
| db.unicorns.find({name: "Ayna"})
|
| [
|   {
|     _id: ObjectId('6a184bf123a5faabc9abc11a'),
|     name: 'Ayna',
|     loves: [ 'strawberry', 'lemon' ],
|     weight: 800,
|     gender: 'f',
|     vampires: 51
|   }
| ]
```

Влюбляем Raleigh в редбул:

```

learn> db.unicorns.updateOne(
|   {name: "Raleigh"},
|   {$addToSet: {loves: "redbull"}}
| )
|
| db.unicorns.find({name: "Raleigh"})
|
| [
|   {
|     _id: ObjectId('6a184bf123a5faabc9abc11c'),
|     name: 'Raleigh',
|     loves: [ 'apple', 'sugar', 'redbull' ],
|     weight: 421,
|     gender: 'm',
|     vampires: 2
|   }
| ]

```

Увеличиваем всем самцам vampires на +5:

```

learn> db.unicorns.updateMany(
|   {gender: "m"},
|   {$inc: {vampires: 5}}
| )
|
| db.unicorns.find({gender: "m"})
|
| [
|   {
|     _id: ObjectId('6a184bf123a5faabc9abc115'),
|     name: 'Horny',
|     loves: [ 'carrot', 'papaya' ],
|     weight: 600,
|     gender: 'm',
|     vampires: 68
|   },
|   {
|     _id: ObjectId('6a184bf123a5faabc9abc117'),
|     name: 'Unicrom',
|     loves: [ 'energon', 'redbull' ]
|   }
| ]

```

Меняем мэра Портланда на беспартийного:

```
learn> db.towns.updateOne(
|   {name: "Portland"},
|   {$unset: {"mayor.party": ""}}
| )
|
| db.towns.find({name: "Portland"})
|
| [
|   {
|     _id: ObjectId('6a18523e23a5faabc9abc123'),
|     name: 'Portland',
|     population: 528000,
|     last_sensus: ISODate('2009-07-20T00:00:00.000Z'),
|     famous_for: [ 'beer', 'food' ],
|     mayor: { name: 'Sam Adams' }
|   }
| ]
```

Влюбляем Pilot в шоколад:

```
learn> db.unicorns.updateOne(
|   {name: "Pilot"},
|   {$addToSet: {loves: "chocolate"}}
| )
|
| db.unicorns.find({name: "Pilot"})
|
| [
|   {
|     _id: ObjectId('6a184bf123a5faabc9abc11e'),
|     name: 'Pilot',
|     loves: [ 'apple', 'watermelon', 'chocolate' ],
|     weight: 650,
|     gender: 'm',
|     vampires: 59
|   }
| ]
```

А аврору в sugar и lemon:

```
learn> db.unicorns.updateOne(
|   {name: "Aurora"},
|   {$addToSet: {loves: {$each: ["sugar", "lemon"]}}}
| )
|
| db.unicorns.find({name: "Aurora"})
|
| [
|   {
|     _id: ObjectId('6a184bf123a5faabc9abc116'),
|     name: 'Aurora',
|     loves: [ 'carrot', 'grape', 'sugar', 'lemon' ],
|     weight: 450,
|     gender: 'f',
|     vampires: 43
|   }
| ]
```

8. Удаление данных из town

Пересоздаем towns и удаляем документы с беспартийными мэрами:

```
learn> db.towns.deleteMany({
|   "mayor.party": {$exists: false}
| })
{ acknowledged: true, deletedCount: 1 }
```

Проверка

```
learn> db.towns.find()
[
  {
    _id: ObjectId('6a18555e23a5faabc9abc126'),
    name: 'New York',
    population: 22200000,
    last_sensus: ISODate('2009-07-31T00:00:00.000Z'),
    famous_for: [ 'statue of liberty', 'food' ],
    mayor: { name: 'Michael Bloomberg', party: 'I' }
  },
  {
    _id: ObjectId('6a18555e23a5faabc9abc127'),
    name: 'Portland',
    population: 528000,
    last_sensus: ISODate('2009-07-20T00:00:00.000Z'),
    famous_for: [ 'beer', 'food' ],
    mayor: { name: 'Sam Adams', party: 'D' }
  }
]
```

Очищаем коллекцию полностью:

```
learn> db.towns.deleteMany({})
{ acknowledged: true, deletedCount: 2 }
learn> db.towns.find()

learn> show collections
towns
unicorns
```

9. Ссылки между коллекциями

Создаем коллекцию habitats:

```
learn> db.habitats.insertMany([
  {
    _id: "forest",
    name: "Forest zone",
    description: "Wooded habitat with rivers and grass fields"
  },
  {
    _id: "mountain",
    name: "Mountain zone",
    description: "Highland habitat with rocks and cold climate"
  },
  {
    _id: "lake",
    name: "Lake zone",
    description: "Habitat near lakes and wet meadows"
  }
])
{
  acknowledged: true,
  insertedIds: { '0': 'forest', '1': 'mountain', '2': 'lake' }
}
```

Добавить ссылки нескольким единорогам:

```

learn> db.unicorns.updateOne(
|   {name: "Aurora"},
|   {$set: {habitat: {$ref: "habitats", $id: "forest"}}}
| )
|
| db.unicorns.updateOne(
|   {name: "Unicrom"},
|   {$set: {habitat: {$ref: "habitats", $id: "mountain"}}}
| )
|
| db.unicorns.updateOne(
|   {name: "Leia"},
|   {$set: {habitat: {$ref: "habitats", $id: "lake"}}}
| )
|
| {
|   acknowledged: true,
|   insertedId: null,
|   matchedCount: 1,
|   modifiedCount: 1,
|   upsertedCount: 0
| }

```

Проверка:

```

learn> db.unicorns.find(
|   {habitat: {$exists: true}},
|   {_id: 0, name: 1, habitat: 1}
| )
|
| [
|   { name: 'Aurora', habitat: DBRef('habitats', 'forest') },
|   { name: 'Unicrom', habitat: DBRef('habitats', 'mountain') },
|   { name: 'Leia', habitat: DBRef('habitats', 'lake') }
| ]

```

Получаем документ зоны по ссылке:

```

learn> var aurora = db.unicorns.findOne({name: "Aurora"})
|
| db.habitats.findOne({_id: "forest"})
|
| {
|   _id: 'forest',
|   name: 'Forest zone',
|   description: 'Wooded habitat with rivers and grass fields'
| }

```


10. Индексы с флагом unique

Посмотрим текущие индексы и создадим индекс по имени:

```
learn> db.unicorns.getIndexes()
[ { v: 2, key: { _id: 1 }, name: '_id_' } ]
learn> db.unicorns.createIndex(
|   {name: 1},
|   {unique: true}
| )
name_1
learn> db.unicorns.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { name: 1 }, name: 'name_1', unique: true }
]
learn> db.unicorns.insertOne({name: "Aurora"})
MongoServerError: E11000 duplicate key error collection: learn.unicorns index: name_1 dup key: { name:
"Aurora" }
```

11. Управление индексами

Удалим все кроме _id_:

```
learn> db.unicorns.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { name: 1 }, name: 'name_1', unique: true }
]
learn> db.unicorns.dropIndexes()
{
  nIndexesWas: 2,
  msg: 'non-_id indexes dropped for collection',
  ok: 1
}
learn> db.unicorns.getIndexes()
[ { v: 2, key: { _id: 1 }, name: '_id_' } ]
```

Попытаемся удалить id:

```
learn> db.unicorns.dropIndex("_id_")
MongoServerError[InvalidOptions]: cannot drop _id index
```

Он системный и обязательный для коллекции.

12. План запроса и сравнение индекса

Создаем коллекцию numbers:

```
learn> db.numbers.drop()
|
| for (let i = 0; i < 100000; i++) {
|   db.numbers.insertOne({value: i})
| }
|
| {
|   acknowledged: true,
|   insertedId: ObjectId('6a185e1123a5faabc9ad47c8')
| }
}
```

Выбираем 4 последних документа:

```
learn> db.numbers.countDocuments()
100000
learn> db.numbers.find().sort({value: -1}).limit(4)
[
  { _id: ObjectId('6a185e1123a5faabc9ad47c8'), value: 99999 },
  { _id: ObjectId('6a185e1123a5faabc9ad47c7'), value: 99998 },
  { _id: ObjectId('6a185e1123a5faabc9ad47c6'), value: 99997 },
  { _id: ObjectId('6a185e1123a5faabc9ad47c5'), value: 99996 }
]
```

План без индекса:

```
learn> db.numbers.explain("executionStats").find().sort({value: -1}).limit(4)
{
  explainVersion: '1',
  queryPlanner: {
    namespace: 'learn.numbers',
    parsedQuery: {},
    indexFilterSet: false,
    queryHash: 'AE5753F2',
    planCacheShapeHash: 'AE5753F2',
    planCacheKey: 'AE69D0D4',
    optimizationTimeMillis: 0,
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
    maxScansToExplodeReached: false,
    prunedSimilarIndexes: false,
    winningPlan: {
      isCached: false,
      stage: 'SORT',
      sortPattern: { value: -1 },
      memLimit: 104857600,
      limitAmount: 4,
      type: 'simple',
      inputStage: { stage: 'COLLSCAN', nss: 'learn.numbers', direction: 'forward' }
    },
    rejectedPlans: []
  },
  executionStats: {
    executionSuccess: true,
    executionTimeMillis: 86,
    totalDocsExamined: 100000,
    ...
  }
}
```

Находим нужные строки:

```
executionTimeMillis: 86,
totalDocsExamined: 100000,
```

Создаем индекс и выполняем снова:

```
learn> db.numbers.createIndex({value: 1})
value_1
learn> db.numbers.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { value: 1 }, name: 'value_1' }
]
learn> db.numbers.find().sort({value: -1}).limit(4)
[
  { _id: ObjectId('6a185e1123a5faabc9ad47c8'), value: 99999 },
  { _id: ObjectId('6a185e1123a5faabc9ad47c7'), value: 99998 },
  { _id: ObjectId('6a185e1123a5faabc9ad47c6'), value: 99997 },
  { _id: ObjectId('6a185e1123a5faabc9ad47c5'), value: 99996 }
]
```

План с индексом:

```
executionTimeMillis: 0,
totalKeysExamined: 4,
totalDocsExamined: 4,
```

Сравнение executionTimeMillis и количества просмотренных документов показало, что запрос с индексом выполняется эффективнее, так как MongoDB может использовать упорядоченную индексную структуру вместо полного просмотра коллекции.

Выводы

В ходе лабораторной работы была установлена и проверена СУБД MongoDB. Было выполнено подключение к локальному серверу через MongoDB Shell. В базе данных learn были созданы коллекции unicorns, towns, habitats и numbers.

В процессе работы были изучены основные операции MongoDB: вставка документов, выборка данных, сортировка, ограничение результата, проекция полей, изменение и удаление документов. Были использованы логические операторы \$gte, \$lte, \$all, \$exists, а также операторы изменения \$set, \$inc, \$unset, \$addToSet и \$each.

Также были выполнены запросы к вложенным объектам с использованием точечной записи, агрегированные запросы с применением countDocuments(), distinct() и aggregate(). Для демонстрации связей между коллекциями были использованы ссылки через \$ref и \$id.

В заключительной части работы были созданы и удалены индексы, а также выполнено сравнение плана запроса до и после создания индекса. Было показано, что индекс по полю `value` позволяет выполнять запрос эффективнее, так как MongoDB может использовать индексную структуру вместо полного просмотра коллекции.